Intermediate (test a tool without calling the LLM) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `tool_test_no_llm_practice.py`: an agent bug is usually one of exactly two things — "the tool itself did the wrong thing" or "the model called the wrong tool." Testing the tool directly, by calling it yourself with no model anywhere in the loop, settles the first question for free, instantly, every single time. **If not:** every tool bug would only show up through the full agent, mixed up with the model's own choices, and cost a real API call each time you tried to check.

Run it with `pytest tool_test_no_llm_practice.py -v`.

Solution

Story: a model will sometimes call a tool that fails. Doc06's `flaky_lookup` catches its own error internally and returns an `"Error: ..."` string as its normal result, so the agent can read it and decide how to recover, instead of crashing. A test checking for that exact string locks this behavior in place for good. **If not:** someone could later "tidy up" the tool so it raises instead of returning an error string, and the agent would start crashing mid-run with no test anywhere to warn you.

```
# practice/tool_test_no_llm_practice.py
# Doc06's tools, inline so this test runs on its own.
from langchain_core.tools import tool


@tool
def add(a: int, b: int) -> int:
    """Add two integers and return the result."""
    return a + b


@tool
def flaky_lookup(query: str, should_fail: bool = False) -> str:
    """Look up a query; catch its own error so the agent can recover."""
    try:
        if should_fail:
            raise RuntimeError("simulated failure")
        return f"Result for {query}"
```

```
    except RuntimeError as error:
        return f"Error: {error}"


def test_add_returns_the_sum():
    # why: pure math — exact == is the right check
    assert add.invoke({"a": 2, "b": 3}) == 5
    assert add.invoke({"a": -1, "b": 1}) == 0

def test_flaky_lookup_success():
    result = flaky_lookup.invoke({"query": "refund policy"})
    assert result == "Result for refund policy"

def test_flaky_lookup_failure():
    # how: should_fail=True forces the failure on purpose — no
    # waiting for a real outage to see what the tool does
    result = flaky_lookup.invoke(
        {"query": "refund policy", "should_fail": True}
    )
    # why: the tool must hand back a readable error, not crash
    assert result == "Error: simulated failure"
```

Expected output:

```
tool_test_no_llm_practice.py::test_add_returns_the_sum PASSED
tool_test_no_llm_practice.py::test_flaky_lookup_success PASSED
tool_test_no_llm_practice.py::test_flaky_lookup_failure PASSED
```